

List Scheduling Algorithm Based on Pre-scheduling for Heterogeneous Computing

Yi Zhao^{1,2}, Suzhi Cao¹ and Lei Yan¹

1. Key Laboratory of Space Utilization, Technology and Engineering Center for Space Utilization, Chinese Academy of Sciences. 100094, Beijing, China.

2. University of Chinese Academy of Sciences, 100049, Beijing, China.

Email: zhaoyi17@mailsucas.edu.cn, {caosuzhi*, yanlei}@csu.ac.cn

Abstract—Efficient application scheduling algorithms are important for heterogeneous computing systems to achieve high performance. List scheduling is a type of classical and widely used algorithm for static application scheduling, with low time complexity but limited result optimization. In this paper, we proposed a novel pre-scheduling-based list scheduling (PSLS) algorithm for heterogeneous computing systems. The PSLS algorithm has the same low time complexity with classical list scheduling algorithms HEFT and PEFT, but significantly reduces the makespans of applications. The PSLS algorithm consists of three phases: pre-scheduling phase, task prioritizing phase and processor selection phase. The simulation results based on randomly generated graphs and graphs of real-world applications show that the PSLS algorithm has obvious optimizations in terms of schedule length, efficiency and robustness compared with classical list scheduling algorithms.

Index Terms—List scheduling, Application scheduling, Heterogeneous computing system, DAG scheduling.

I. INTRODUCTION

A group of heterogeneous computing resources connected by high-speed network constitute a heterogeneous computing system [1], [2]. Such a system can support fast processing of computation-intensive parallel applications. Heterogeneous computing has been applied into cloud computing, edge computing, grid computing and other environments. With the developments of big data, artificial intelligence and Internet of Things, there are more requirements for heterogeneous computing to accelerate data processing [1], [3], [4].

In heterogeneous computing systems, effective scheduling for applications is a key factor to attain high performance and realize efficient utilization of resources. In general, a computationally intensive application can be decomposed into a set of tasks with precedence constraints, represented as a directed acyclic graph (DAG). In static scheduling model, each task has a certain amount of data to process, and there is a certain amount of data transmitted between two tasks. Different processors have different execution speed for these tasks. Hence the target of scheduling algorithms is to arrange each task on the appropriate processor to minimize the total completion time of application, called *makespan*.

The application scheduling problem is actually a DAG scheduling problem, which has been proved to be a NP-complete problem [5]. The existing static scheduling algorithms for heterogeneous systems can be classified into

two categories: heuristic-based algorithms and guided random search algorithms.

The heuristic-based scheduling algorithms are mainly of 3 types: list scheduling [6]–[10], cluster scheduling [11] and duplication scheduling [12]. Most of heuristic-based algorithms belong to list scheduling. There are two phases of list scheduling: task prioritizing phase and processor selection phase. Different list scheduling algorithms have different rules in these two phases. Algorithms HEFT [6] and PEFT [10] are two classical list scheduling algorithms with $O(n^2 \cdot m)$ time complexity. In general, list scheduling can generate a good solution within polynomial time since the search is narrowed down to a small portion of the solution space. But this is also why the optimization of the solution is still limited.

Is there a method to achieve better solution while maintaining the low time complexity? In this paper, we proposed a novel pre-scheduling-based list scheduling (PSLS) algorithm for heterogeneous computing systems. The PSLS algorithm has the same low time complexity $O(n^2 \cdot m)$ with classical list scheduling algorithms, but significantly reduces the *makespans* of applications. The PSLS algorithm consists of three phases: pre-scheduling phase, task prioritizing phase and processor selection phase.

The key contributions of our work are listed as follows:

- We proposed a new pre-scheduling phase, which is the key to improve the scheduling performance. The method of pre-scheduling is universal and effective to different algorithms.
- We proposed an important concept downward length table (DLT) and DLT-related heuristic scheduling rules.
- The simulation was done to verify the performance of PSLS. The result shows that the PSLS algorithm can attain better schedule length, efficiency and robustness compared with classical list scheduling algorithms.

This paper is organized as follows: in Section II, we model the application scheduling problem; in Section III, we introduce some related work of list scheduling; in Section IV, we propose our pre-scheduling-based list scheduling (PSLS) algorithm; in Section V, we compare the performance of PSLS with other classical list scheduling algorithms; finally, we give a summary of our work and a future research prospect in Section VI.

II. SCHEDULING PROBLEM MODEL

A. Application Model

As shown in Fig. 1(a), a computationally intensive application can be represented by a directed acyclic graph (DAG) [6]–[10], $G=(V, E)$, where V is the set of n tasks and E is the set of e edges between tasks.

Directed edge from task n_i to task n_j , $e(i, j)$, represents the precedence constraint of n_i and n_j . The output of task n_i is part of the input of task n_j , therefore task n_i must complete its execution before task n_j can be started. Task n_i may have several immediate successors, whose set is denoted as $succ(n_i)$. And Task n_i may also have several immediate predecessors, whose set is denoted as $pred(n_i)$. A task without immediate successor is an exit task, denoted as n_{exit} . A task without immediate predecessor is an entry task, denoted as n_{entry} .

Task n_i has a certain amount of data to process, denoted as D_i . Between task n_i and task n_j , there is a certain amount of data to be transmitted, denoted as $D_{i,j}$.

See Table I for all definitions of notations in Section II.

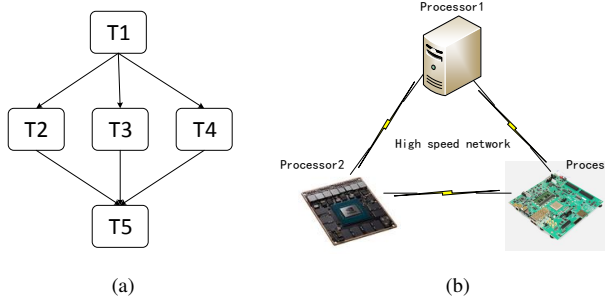


Fig. 1. (a) An application DAG with 5 tasks. (b) A heterogeneous computing system with 3 processors.

B. Heterogeneous Computing Model

As shown in Fig. 1(b), a heterogeneous computing system P consists of m heterogeneous processors, which are fully connected by a high-speed network. The k th processor is denoted as p_k .

In heterogeneous computing system, the execution speed of task n_i on processor p_k is denoted as $s_{i,k}$. The computation cost $w_{i,k}$, i.e. the estimated time to execute n_i on p_k is given by (words computation cost and execution time of task have the same meaning in this paper):

$$w_{i,k} = D_i / s_{i,k} \quad (1)$$

The communication rate from processor p_k to processor p_l is denoted as $B_{k,l}$. When task n_i is arranged to processor p_k and task n_j is arranged to processor p_l , the communication cost $c_{i,j}$, i.e. the estimated time to transmit the data between n_i and n_j from p_k to p_l is given by (words communication cost and transmission time have the same meaning in this paper):

$$c_{i,j} = L + D_{i,j} / B_{k,l} \quad (2)$$

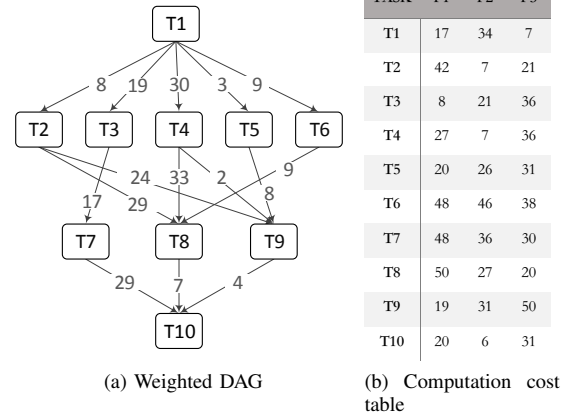


Fig. 2. A sample of application weighted DAG and computation cost table.

Where L is the communication startup time.

In heterogeneous computing environment, the characteristics of an application can be represented by the weights of nodes and edges in DAG (see Fig. 2a). The weight of task n_i is the mean execution time of task n_i , which is calculated as follows:

$$\bar{w}_i = (\sum_{p_k \in P} w_{i,k}) / m \quad (3)$$

And the weight of edge $e(i, j)$ is the mean transmission time between n_i and n_j , which is calculated as follows:

$$\bar{c}_{i,j} = \bar{L} + D_{i,j} / \bar{B} \quad (4)$$

Where $\bar{L}$ is the average communication startup time of all processors and $\bar{B}$ is the average communication rate of all links connecting processors.

For an application in a specific heterogeneous computing system, the communication to computation ratio (ccr) [6]–[10] is the average communication cost divided by the average computation cost. It can be used to indicate whether a task graph is communication-intensive or computation-intensive. The ccr is given by:

$$ccr = \frac{\frac{1}{e} \sum_{e(i,j) \in E} \bar{c}_{i,j}}{\frac{1}{n} \sum_{n_i \in V} \bar{w}_i} \quad (5)$$

C. Task Scheduling Model

Scheduling algorithms for heterogeneous computing systems take application's weighted DAG (see Fig. 2(a)) and computation cost table (see Fig. 2(b)) as inputs. The computation cost table is a matrix with the size of $n \times m$. The element in column k of line i is the computation cost of task n_i on processor p_k , $w_{i,k}$. According to these inputs, scheduling algorithms follow some rules to select processor for each task.

Next, we introduce some basic concepts used in task scheduling [6]–[10]. It is worth noting that these concepts are defined recursively, but will not fall into endless cycles. Before scheduling, the default initial values of these concepts are 0.

TABLE I
DEFINITIONS OF NOTATIONS

Notations	Definitions
V	The set of tasks
E	The set of edges between tasks
P	The heterogeneous computing system
$\text{succ}(n_i)$	The set of immediate successors of n_i
$\text{pred}(n_i)$	The set of immediate predecessors of n_i
n_i	The i th task
p_k	The k th processor
n	The number of tasks
m	The number of processors
n_{exit}	The exit task
n_{entry}	The entry task
$\bar{B}$	The average bandwidth
$\bar{L}$	The average communication startup time
D_i	The amount of data to process for task n_i
$D_{i,j}$	The amount of data transferred from n_i to n_j
$e(i,j)$	The edge from n_i to n_j
$s_{i,k}$	The processing speed of n_i on p_k
$w_{i,k}$	The computation cost of n_i on p_k
$B_{k,l}$	The bandwidth from p_k to p_l
$c_{i,j}$	The average communication cost from n_i to n_j
ccr	The communication to computation ratio

In scheduling, these values are updated iteratively. When all tasks are scheduled, these values stop updating.

Definition 1. Earliest Start Time (EST). Task n_i can be executed on processor p_k only when p_k is ready and all immediate predecessor tasks have completed execution and data transmission to p_k . The earliest start time (EST) of task n_i on processor p_k is defined as follows:

$$EST(n_i, p_k) = \max \left\{ Avail(p_k), \max_{n_j \in \text{pred}(n_i)} \{AFT(n_j) + c_{i,j}\} \right\} \quad (6)$$

See $AFT(n_j, p_l)$ in Definition 7 and $Avail(p_k)$ in Definition 2. The communication cost $c_{i,j}$ is zero if the task n_j is assigned to processor p_k . For the entry task on any processor, $EST(n_{\text{entry}}, p_k) = Avail(p_k)$.

Definition 2. Earliest Available Time (Avail). The earliest available time (Avail) of the processor p_k equals the actual end time (AFT) of the last task that has been allocated on p_k . Avail of p_k is calculated as follows:

$$Avail(p_k) = \max_{\text{map}(n_j)=p_k} AFT(n_j) \quad (7)$$

See $\text{map}(n_j)$ in Definition 5.

Definition 3. Earliest Finish Time (EFT). The earliest finish time (EFT) of task n_i on processor p_k equals $EST(n_i, p_k)$ plus the computation cost of n_i on p_k , $w_{i,k}$. The $EFT(n_i, p_k)$ is defined as follows:

$$EFT(n_i, p_k) = EST(n_i, p_k) + w_{i,k} \quad (8)$$

Definition 4. Optimization Objective Function (OPT). When scheduling each task, the scheduler iterates through all processors and calculates the value of the optimization objective

function $OPT(n_i, p_k)$ assuming that the current task n_i is arranged on processor p_k . Finally, task is arranged on the processor with minimum objective function values.

Definition 5. Task-processor map (map). The processor of n_i is selected according to the rule of minimization of OPT . The processor of task n_i is denoted as follows:

$$\text{map}(n_i) = \arg \min_{p_k \in P} OPT(n_i, p_k) \quad (9)$$

Definition 6. Actual Start Time (AST). Once the task n_i is scheduled and its processor is determined, the actual start time (AST) of task n_i is determined by:

$$AST(n_i) = EST(n_i, \text{map}(n_i)) \quad (10)$$

Definition 7. Actual Finish Time (AFT). Once the task n_i is scheduled and its processor is determined, the actual finish time (AFT) of task n_i is determined by:

$$AFT(n_i) = EFT(n_i, \text{map}(n_i)) \quad (11)$$

Definition 8. Total Execution Time (makespan). Words total completion time, makespan and scheduling length have the same meaning in this paper. The makespan is defined by:

$$\text{makespan} = \max \{AFT(n_{\text{exit}})\} \quad (12)$$

Definition 9. Critical Path (CP). The critical path (CP) is the longest path from the entry tasks to the exit tasks in DAG.

III. RELATED WORK

List scheduling algorithms usually have two phases: task prioritizing phase and processor selection phase. In task prioritizing phase, the scheduler calculates the priorities of application's tasks according to a certain priority function. Then a priority task queue is generated by descending priority values. The task queue determines the order of task scheduling. In processor selection phase, scheduler select the processor for each task in turn by minimizing the OPT. Next, we introduce several representative list scheduling algorithms, including HEFT, PEFT, Lookahead, CPOP, HCPT and SDBATS. Table II gives a comparison of these algorithms' time complexity, where n denotes the number of tasks and m denotes the number of processors.

The Heterogeneous Earliest Finish Time (HEFT) algorithm [6] is one of the earliest list scheduling algorithms proposed. It's also one of the most famous and widely used algorithms [13]. The HEFT algorithm has two phases. HEFT's task priority function is upward ranking and its OPT is $EFT(n_i, p_k)$.

The Predict Earliest Finish Time (PEFT) algorithm [10] has the same time complexity $O(n^2 \cdot m)$ with HEFT, but offering makespan reduction by introducing an optimistic cost table (OCT). The upward ranking can be chosen as PEFT's task priority function. The OPT function of PEFT algorithm is the sum of $EFT(n_i, p_k)$ and $OCT(n_i, p_k)$.

The Lookahead algorithm [8] usually can achieve better result than the HEFT algorithm while bringing a higher time complexity $O(n^4 \cdot m^3)$. Lookahead's task priority function is upward ranking and its OPT is the EFT of current task's

TABLE II
COMPARISON OF LIST SCHEDULING ALGORITHMS' TIME COMPLEXITIES

Algorithm	Reference	Complexity
HEFT	[6]	$O(n^2 \cdot m)$
CPOP	[6]	$O(n^2 \cdot m)$
HCPT	[7]	$O(n^2 \cdot m)$
Lookahead	[8]	$O(n^4 \cdot m^3)$
SDBATS	[9]	$O(n^2 \cdot m)$
PEFT	[10]	$O(n^2 \cdot m)$

immediate successors. This algorithm need more time to execute scheduling, especially when the scale of application or the scale of heterogeneous computing system is large. The high time complexity is the limitation of Lookahead algorithm.

The CPOP algorithm's [6] task priority function is the sum of upward ranking and downward ranking. Processors are selected by minimizing computation cost for tasks in CP and by minimizing the EFT for others. The time complexity of CPOP is same with HEFT algorithm, but the performance of CPOP is worse than that of HEFT.

The concept of CP is also used in HCPT algorithm [7]. The task list is implemented by an empty queue and an auxiliary stack containing tasks in CP. And processor selection is performed by minimizing $EFT(n_i, p_k)$.

The SDBATS algorithm [9] introduces heterogeneous standard deviation parameters to calculate the upward rankings of tasks. And the OPT is $EFT(n_i, p_k)$. Its performance is better than HEFT and CPOP.

IV. THE PSLS ALGORITHM

In this section, we propose the pre-scheduling-based list scheduling (PSLS) algorithm for heterogeneous computing systems. The PSLS algorithm has three phases: pre-scheduling phase, task prioritizing phase and processor selection phase. These three phases are detailed in Sections IV. A, IV. B and IV. C respectively. Then we analysis the PSLS algorithm's time complexity in Section IV. D.

A. Pre-scheduling Phase

The first phase of the PSLS algorithm is the pre-scheduling phase. The goal in this phase is to produce a real "task-processor" scheduling result, which will be used in the second phase.

In order to get a complete scheduling result, we need to choose an existing scheduling algorithm as pre-scheduling algorithm. In theory, any scheduling algorithm can be used as a pre-scheduling algorithm and provide sufficient input for the second phase. But because we want to improve algorithm performance while maintaining low time complexity, we should choose pre-scheduling algorithm from algorithms with time complexity not higher than $O(n^2 \cdot m)$. Algorithms with a higher time complexity will not be considered. According to plenty experimental results (see the result in Fig. 3 and the definition of SLR in Section V) of several list scheduling

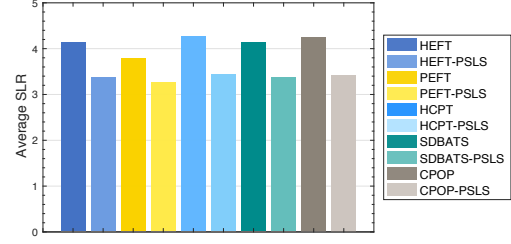


Fig. 3. Performance comparison of different pre-scheduling algorithms and their corresponding original algorithms.

algorithms with $O(n^2 \cdot m)$ complexity, including HEFT, CPOP, HCPT, SDBATS and PEFT, we found that the improvement effect of pre-scheduling method is universal and effective to different algorithms. It means that no matter which algorithm is used in the pre-scheduling phase, the average scheduling length ratio is much less than that of the original algorithm. We finally choose the PEFT algorithm as our pre-scheduling algorithm, using upward ranking as PEFT's priority function.

The pre-scheduling results include "task-processor" map map_1 , actual start time of tasks AST_1 and pre-scheduling makespan $makespan_1$. The map_1 is a vector of length n . $map_1(n_i) = p_k$ means that in pre-scheduling task n_i has been assigned to processor p_k . The AST_1 is a vector of length n . $AST_1(n_i) = t$ means that task n_i starts actually at time t . The $makespan_1$ is the total completion time of application according to the pre-scheduling results. The map_1 , AST_1 and $makespan_1$ can already form a complete scheduling result. However, in the PSLS algorithm, we don't output them directly. These results will be used in the second phase to calculate the downward length table (DLT). The pre-scheduling results of the sample DAG in Fig. 2 are shown in Table III.

B. Task Prioritizing Phase

The second phase of the PSLS algorithm is task prioritizing phase, in which the priorities of all tasks are calculated. The goal of this phase is to generate a task priority queue with descending priorities, determining the order of task scheduling in next phase.

Here we proposed an important concept: downward length table (DLT). The DLT is a matrix with the size of $n \times m$. The content of the DLT is calculated using the weighted DAG and the pre-scheduling results in the first phase, including map_1 , AST_1 and $makespan_1$. In DLT, The element in column k of line i is denoted as $DLT(n_i, p_k)$. $DLT(n_i, p_k)$ is the difference between pre-scheduling makespan and the latest finish time of task n_i on processor p_k according to the DAG weighted precedence constraints, when all immediate successors of n_i has been arranged as pre-scheduling results. $DLT(n_i, p_k)$ is calculated by:

$$DLT(n_i, p_k) = makespan_1 - \max_{n_j \in succ(n_i)} \{AST_1(n_j) - \overline{c_{i,j}}\} \quad (13)$$

TABLE III
PHASED RESULTS OF THE SAMPLE DAG IN FIG. 2 USING PSLS ALGORITHM

1. Pre-scheduling phase				2. Task prioritizing phase						3. Processor selection phase				
Task	map_1	AST_1	$makespan_1$	Task	DLT			$rank_{DLT}$	Priority	Priority	OPT(EFT+DLT)			Processor
T1	3	0	120	T1	121	121	113	118.3	T1	T1	138	155	120	3
T2	3	7		T2	64	88	88	80	T2	T2	121	110	116	2
T3	2	26		T3	83	66	83	77.3	T3	T3	117	113	126	2
T4	2	47		T4	66	66	66	66	T5	T5	94	145	110	1
T5	1	10		T5	64	72	72	69.3	T4	T4	130	120	109	3
T6	3	28		T6	42	42	33	39	T6	T6	120	135	114	3
T7	2	54		T7	35	6	35	25.3	T7	T7	147	89	146	2
T8	3	87		T8	13	6	13	10.7	T8	T8	153	123	114	3
T9	1	56		T9	10	6	10	8.7	T9	T9	75	120	161	1
T10	2	114		T10	0	0	0	0	T10	T10	132	114	143	2

Where the communication cost $c_{i,j}$ equals zero if the n_j is assigned to processor p_k . On the right side of the equation, p_k is an implicit variable, because for the exit task on any processor, $DLT(n_{exit}, p_k) = 0$.

We calculated the DLT before task prioritizing. Then we use the DLT to calculate priority value $rank_{DLT}$ for every task, shown as follows:

$$rank_{DLT}(n_i) = \frac{1}{m} \sum_{k=1}^m DLT(n_i, p_k) \quad (14)$$

The second phase results of the sample DAG in Fig. 2 are shown in Table III.

C. Processor Selection Phase

The third phase of the PSLS algorithm is processor selection phase, in which scheduler choose processor for each task in the priority queue. The optimization objective function is defined as follows:

$$OPT_{PSLS}(n_i, p_k) = EFT(n_i, p_k) + DLT(n_i, p_k) \quad (15)$$

The DLT brings lookahead information based on pre-scheduling to the optimization objective function, which is the most important reason for the performance improvement of the PSLS algorithm.

The third phase results of the sample DAG in Fig. 2 are shown in Table III.

D. Description of PSLS Algorithm

The proposed pre-scheduling-based list scheduling (PSLS) algorithm has three phases: pre-scheduling phase, task prioritizing phase and processor selection phase. In the pre-scheduling phase, this algorithm uses the PEFT algorithm to get certain pre-scheduling results. In the task prioritizing phase, this algorithm calculates the DLT using the weighted DAG and pre-scheduling results. The task priority queue is generated according to $rank_{DLT}$. In the processor selection phase, the algorithm will select processors for tasks in priority queue in turn by minimizing $EFT(n_i, p_k) + DLT(n_i, p_k)$. The proposed PSLS algorithm is formalized in Algorithm 1.

The PSLS algorithm has the same time complexity with algorithms HEFT and PEFT. The pre-scheduling phase of

Algorithm 1 PSLS

```

1: function PSLS(DAG, Computation cost table)
2:   Execute the pre-scheduling PEFT algorithm
3:   Calculate the DLT using pre-scheduling results
4:   Calculate  $rank_{DLT}$  for all tasks
5:   Create task queue with descending  $rank_{DLT}$ 
6:   while task queue is NOT Empty do
7:      $n_i \leftarrow$  the task with highest  $rank_{DLT}$  from task queue;
8:     for all  $p_k$  in P do
9:       Compute  $EFT(n_i, p_k)$  using insertion policy;
10:       $OPT(n_i, p_k) = EFT(n_i, p_k) + DLT(n_i, p_k)$ ;
11:    end for;
12:    Assign task  $n_i$  to  $p_k$  that minimize  $OPT(n_i, p_k)$ ;
13:    Update task queue;
14:  end while
15: end function

```

PSLS has $O(n^2 \cdot m)$ time complexity according to the time complexity of PEFT. In task prioritizing phase, calculation of the DLT has $O(m(n + e))$ time complexity and calculation of the $rank_{DLT}$ for all tasks has $O(m \cdot n)$ time complexity. The processor selection phase of PSLS has $O(n^2 \cdot m)$ time complexity. For dense DAGs, e becomes n^2 , where the total algorithm complexity is of the order $O(n^2 \cdot m)$. That is, the time complexity of PSLS is of the same order as algorithms HEFT and PEFT. In addition to the time complexity comparison, we also show the comparison of the real runtime of these algorithms in Section V. B.

Fig. 4 shows the scheduling results of the sample DAG in Fig. 2 using the algorithms HEFT, Lookahead, PEFT and PSLS. Their *makespans* are 129, 123, 120 and 114 respectively.

V. EXPERIMENTAL RESULTS

In this section, we give a comparison of the performance of the proposed PSLS algorithm with these of algorithms HEFT, PEFT and Lookahead. And we use randomly generated application graphs and real-world application graphs as the simulation input. All of our simulation is based on

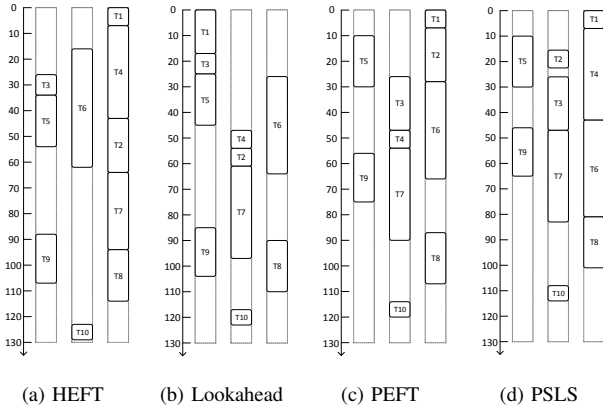


Fig. 4. Schedules of the sample task graph in Fig. 2 with (a) HEFT ($makespan = 129$), (b) Lookahead ($makespan = 123$), (c) PEFT ($makespan = 120$), (d) PSLs ($makespan = 114$).

MATLAB. Simulation related code is accessible in Github: <https://github.com/zhaoyi17/PSLS.git>.

A. Comparison Metrics

Four metrics have been selected for the performance comparison of these algorithms: scheduling length ratio, efficiency, the percentage of occurrences of better solutions and runtime of algorithm itself.

1) *Scheduling Length Ratio (SLR)*: A smaller $makespan$ value means a better performance of scheduling algorithm. The scheduling length ratio (SLR) is the normalized $makespan$, calculated by:

$$SLR = \frac{makespan(solution)}{\sum_{n_i \in CP} \min_{p_k \in P} w_{i,k}} \quad (16)$$

Where the denominator is the minimum computation cost of the critical path tasks. A lower SLR value means a better performance of scheduling algorithm.

2) *Efficiency*: The efficiency is a metric to measure the acceleration of parallel computing to serial computing for a given task scheduling [6], [10], calculated by:

$$Speedup = \frac{\min_{p_k \in P} \{\sum_{n_i \in V} w_{i,k}\}}{makespan(solution)} \quad (17)$$

Where the numerator is the minimum execution time when assigning all tasks to a single processor. A higher $Speedup$ value means a better performance of scheduling algorithm.

3) *Percentage of Occurrences of Better Solutions*: The percentage of better, equal, and worse solutions produced by different algorithms [6], [10].

4) *The Actual Average Runtime*: The real runtime of an algorithm itself [6].

B. Random Graph Generator

We designed and implemented a random graph generator to provide sufficient input for algorithm performance examination. The random graph generator can generate random

application DAGs and corresponding computation cost tables according to a group of given parameters. By changing several parameters, we can get random graphs with different form and weights to evaluate the robustness of scheduling algorithms. Input parameters are as follows:

- n : number of tasks in DAG.
- m : number of processors in heterogeneous computing system.
- α : DAG shape parameter, affecting the height and the width of a DAG.
- β : heterogeneity parameter, affecting the range percentage of computation costs on different processors.
- ccr : The average communication cost to average computation cost ratio, affecting whether an application is communication-intensive or computation-intensive.

C. Random Graph Results

In our experiment, we chose the following parameters for the generation of random DAG :

- $n = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]$
- $m = [4, 8, 16, 32]$
- $\alpha = [0.1, 0.4, 0.8]$
- $\beta = [0.1, 0.2, 0.5, 1, 1.5, 2]$
- $ccr = [0.2, 0.5, 0.8, 1, 2, 5, 10]$

For each combination of parameter values, 100 experiments were executed. And the total number of random graphs is 504000. Assigning several input parameters and selecting each

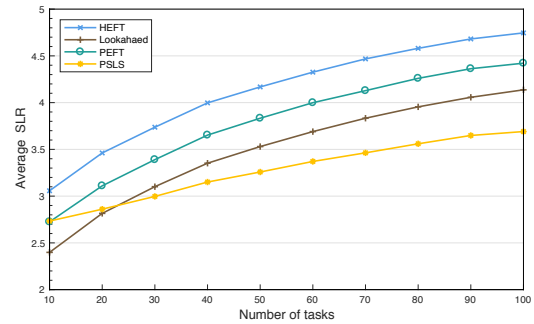


Fig. 5. Average SLR of random graphs for different task numbers

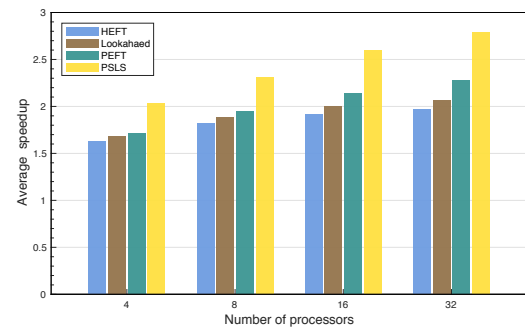


Fig. 6. Average $Speedup$ of random graphs for different processor numbers

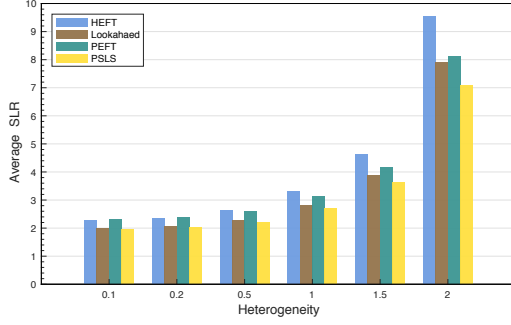


Fig. 7. Average *SLR* of random graphs for different heterogeneity

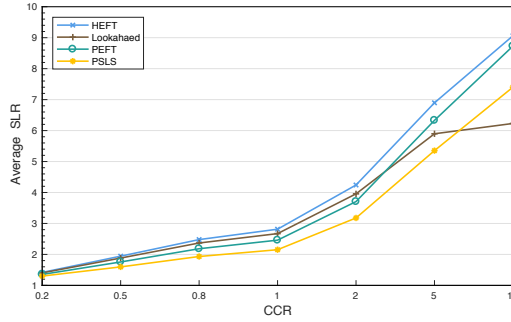


Fig. 8. Average *SLR* of random graphs for different *ccr*

TABLE IV
PAIRWISE SCHEDULE LENGTH COMPARISON OF THE SCHEDULING ALGORITHMS

		HEFT	PEFT	Lookahead
PSLS	better	72.2%	71.1%	60.4%
	equal	1.0%	5.1%	1.4%
	worse	26.8%	23.8%	38.2%
Lookahead	better	72.3%	53.0%	
	equal	4.6%	2.8%	
	equal	23.1%	44.2%	
PEFT	better	60.3%		
	equal	2.3%		
	equal	37.4%		

parameter from a set cause the generation of diverse DAGs, which can prevent biasing of algorithm assessment.

Fig. 5 shows the average *SLR* of these algorithms as a function of application task number. Fig. 6 shows the average *Speedup* of these algorithms as a function of processor number. Fig. 7 shows the average *SLR* of these algorithms as a function of system heterogeneity. Fig. 8 shows the average *SLR* of these algorithms as a function of *ccr*. PSLS has an significant lower *SLR* and a higher *Speedup* than other algorithms, which means a better performance.

Table IV shows the percentage of better, equal, and worse results for these algorithms. Compared with HEFT, PEFT and Lookahead, our PSLS algorithm achieves NOT worse scheduling in 73.2 percent, 76.2 percent and 61.8 percent of

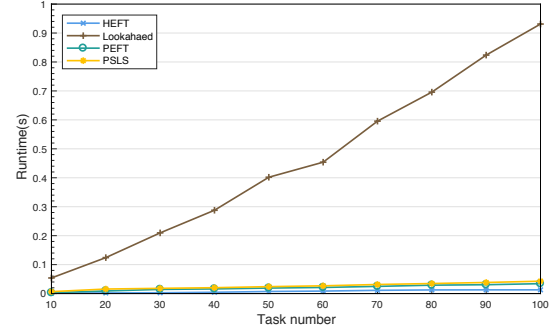


Fig. 9. The comparison of runtimes of different algorithms

runs respectively. Fig. 9 shows the average runtimes of these algorithms respectively when the task number changes from 10 to 100. We can see that compared with HEFT and PEFT, the PSLS's extra runtime required is slight. compared with Lookahead, the PSLS saves a lot of time. According to the analysis of the above simulation results, we conclude that the PSLS algorithm's scheduling result is not only significantly superior to HEFT and PEFT algorithms with $O(n^2 \cdot m)$ time complexity, but also comparable to Lookahead algorithm whose time complexity is $O(n^4 \cdot m^3)$.

D. Real-World Application Results

We also consider two real-world applications as input for the simulation: Fast Fourier Transform (FFT) [14] and Montage [15]. FFT is an efficient and fast method for computing Discrete Fourier Transform (DFT). A sample DAG of FFT is shown in Fig. 10(a). Montage is an application that constructs custom science-grade astronomical image mosaics. A sample DAG of Montage is shown in Fig. 10(b).

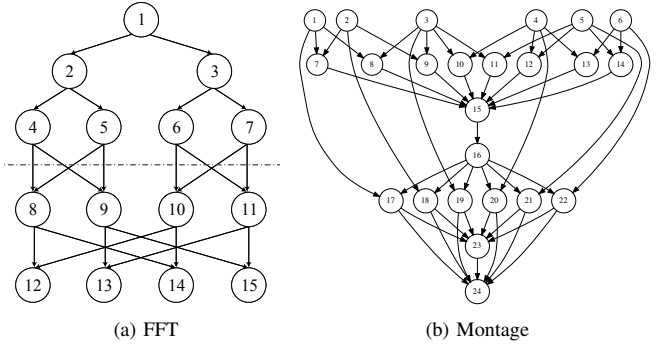


Fig. 10. Application DAGs of FFT and Montage.

For specific applications, the structural features and weights of DAG have been determined, which depend on how this application works. Hence, we only consider part variables.

Fig. 11 shows the average *SLR* of these algorithms as a function of input length of FFT. Fig. 12 shows the average *Speedup* of these algorithms as a function of *ccr* in Montage experiment. PSLS has an significant lower *SLR* than other algorithms.

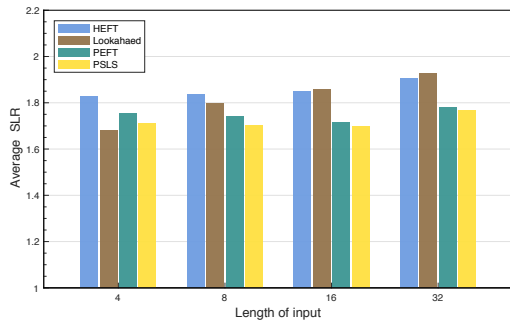


Fig. 11. Average SLR of FFT for different task numbers

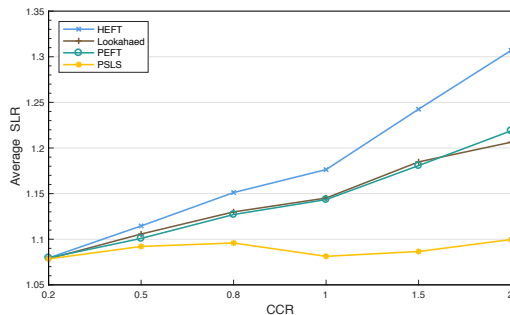


Fig. 12. Average SLR of Montage for different ccr

VI. CONCLUSION

In this paper, we proposed a novel pre-scheduling-based list scheduling (PSLS) algorithm for heterogeneous systems. The PSLS algorithm significantly reduces the makespans of applications and has the same low time complexity with classical list scheduling algorithms. The PSLS algorithm consists of three phases: pre-scheduling phase, task prioritizing phase and processor selection phase. The DLT is an important concept proposed by us. For calculating the DLT, pre-scheduling phase is necessary, which is an additional phase compared with other list scheduling algorithms. Simulation results based on millions of application DAGs with different characteristics show that the PSLS algorithm has remarkable optimizations in schedule length, efficiency and robustness compared with classical list algorithms. The future research direction is the application of PSLS algorithm in special scenes and its engineering implementation.

ACKNOWLEDGMENT

This work was supported by national defense innovation special zone project. This work was supported by the Research Fund of The manned space engineering(No.18022010301). This work has been partially supported by the Open Fund of IPOC (BUPT) and the National Natural Science Foundation of China (No.61701484).

REFERENCES

- [1] S. P. Crago and J. P. Walters, "Heterogeneous cloud computing: The way forward," *Computer*, vol. 48, no. 1, pp. 59–61, 2015.

- [2] A. R. Brodtkorb, C. Dyken, T. R. Hagen, J. M. Hjelmervik, and O. O. Storaasli, "State-of-the-art in heterogeneous computing," *Scientific Programming*, vol. 18, no. 1, pp. 1–33, 2010.
- [3] W. Wei, B. Liang, and B. Li, "Multi-resource fair allocation in heterogeneous cloud computing systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 26, no. 10, pp. 2822–2835, 2015.
- [4] S. P. Crago, K. Dunn, P. Eads, L. Hochstein, and J. P. Walters, "Heterogeneous cloud computing," in *IEEE International Conference on Cluster Computing*, 2011.
- [5] J. D. Ullman, "Np-complete scheduling problems *," *Journal of Computer and System Sciences*, vol. 10, no. 3, pp. 384–393, 1975.
- [6] H. Topcuoglu, S. Hariri, and M. Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [7] T. Hagras and J. Janeczek, "A simple scheduling heuristic for heterogeneous computing environments," in *International Conference on Parallel and Distributed Computing*, 2003.
- [8] L. F. Bittencourt, R. Sakellariou, and E. R. M. Madeira, "Dag scheduling using a lookahead variant of the heterogeneous earliest finish time algorithm," in *Euromicro International Conference on Parallel*, 2010.
- [9] E. U. Munir, S. Mohsin, A. Hussain, M. W. Nisar, and S. Ali, "Sdbats: A novel algorithm for task scheduling in heterogeneous computing systems," in *IEEE International Symposium on Parallel and Distributed Processing*, 2013.
- [10] H. Arabnejad and J. G. Barbosa, "List scheduling algorithm for heterogeneous systems by an optimistic cost table," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 3, pp. 682–694, 2014.
- [11] K. Gkoutioudi and H. D. Karatza, "Task cluster scheduling in a grid system," *Simulation Modelling Practice and Theory*, vol. 18, no. 9, pp. 1242–1252, 2010.
- [12] O. Sinnen, A. To, and M. Kaur, "Contention-aware scheduling with task duplication," *Journal of Parallel and Distributed Computing*, vol. 71, no. 1, pp. 77–86, 2011.
- [13] Canon, L. C, et al, "Comparative Evaluation Of The Robustness Of DAG Scheduling Heuristics," *Grid Computing*, 2008.
- [14] S. M Kuo and B. H Lee, *Fast Fourier Transform and Its Applications*, 06 2002, pp. 303–350.
- [15] B. Berriman, "Montage: A grid-enabled image mosaic service for the nvo," 06 2019.